

CS4432: Database Systems II

Data Storage



DreamStudio Lite prompt:
Data hoarder in self-storage

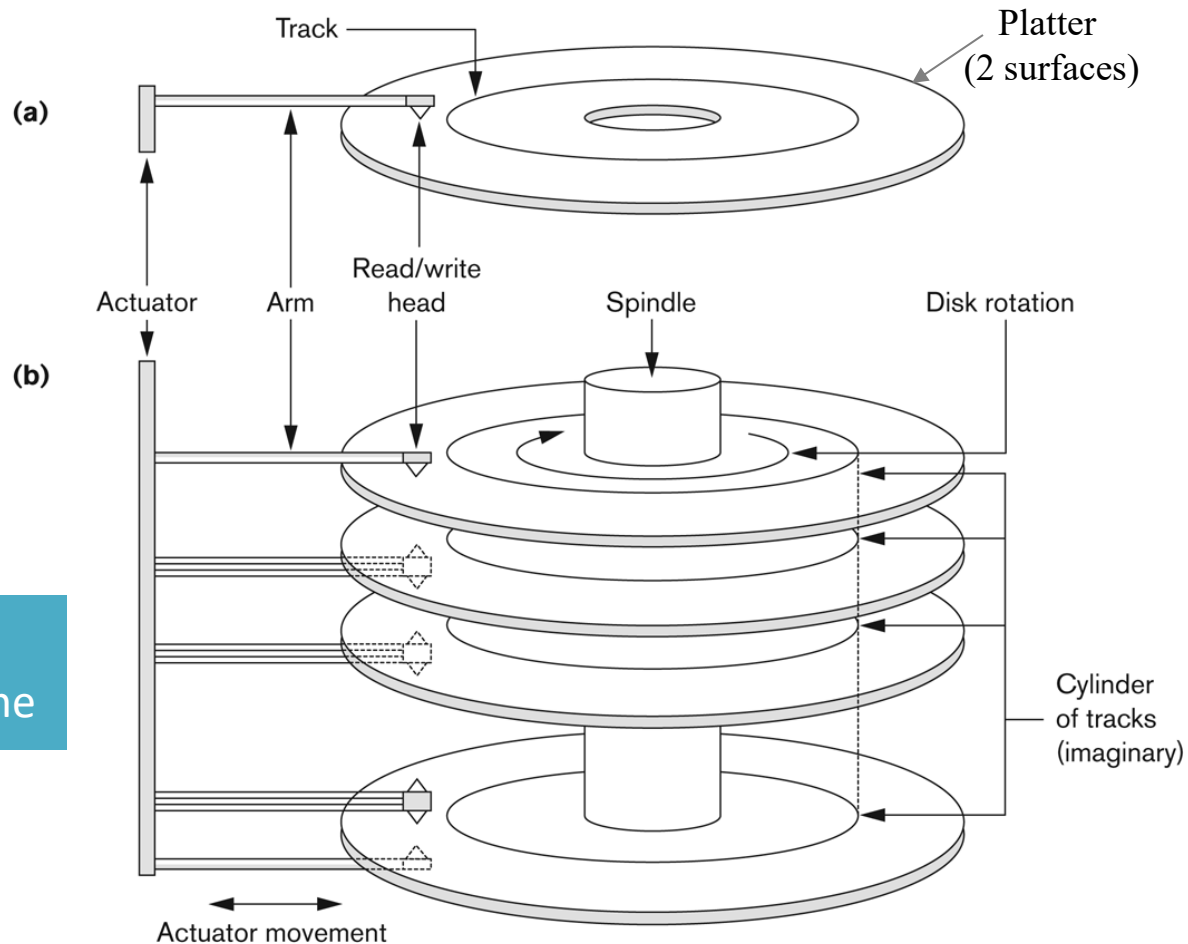
Updates

- Join **Discord Server**
- Monday Mar 24: **Quiz 1 (in-person)**
 - Please come talk to me if you have **accommodations** or cannot make it to class.
- **HW1** is posted
 - **Deadline: Mar 26**

Recap: Disk Components

Figure 13.1

(a) A single-sided disk with read/write hardware. (b) A disk pack with read/write hardware.



Arms move together

Only ONE head can Read or Write at a time

Recap: Tracks divided into Sectors

In the last lecture, we talked about **geometrical sector**.

Today we will focus on **disk sector** (or 'sector' for short)

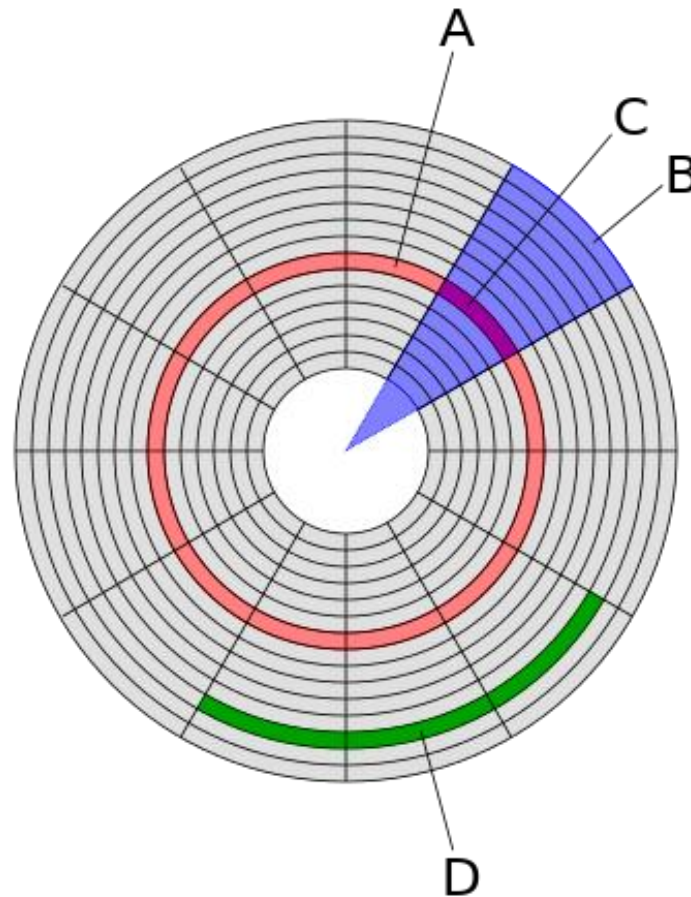
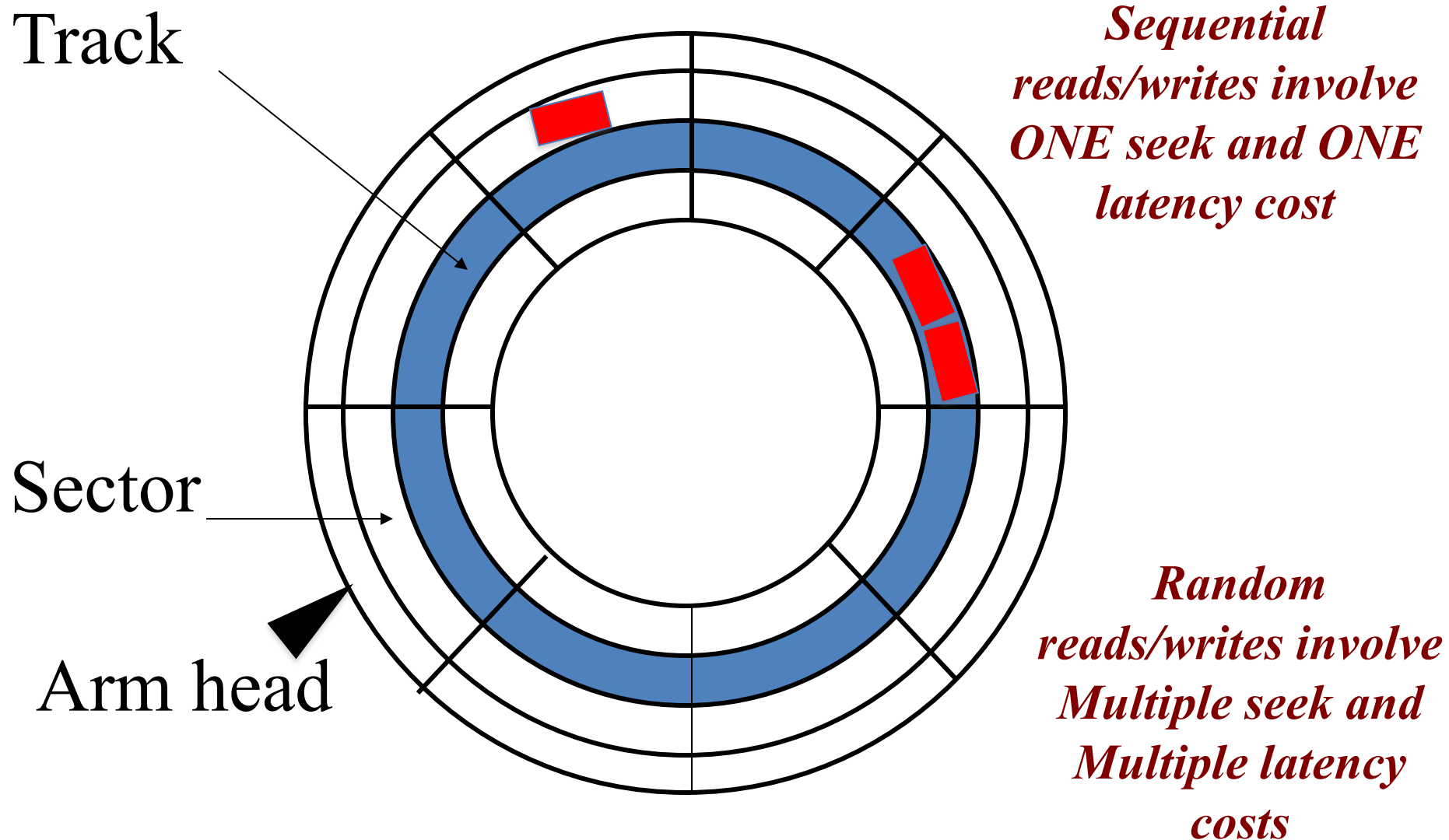


Figure 1: Disk structures:

- (A) Track
- (B) Geometrical sector
- (C) Disk sector
- (D) Cluster

Sequential I/O vs. Random I/O



Learning Objectives

- Compute storage size based on disk specs
- Explain components of disk access time
- Compute MIN, MAX and AVG latency time

How big is the disk if?

Example:

DO NOT FORGET

Each platter has two surfaces

- There are 4 platters
- There are 8192 tracks/surface
- There are 256 sectors/track
- There are 512 bytes/sector

Remember 1KB = 1024 bytes, not 1000!

Size = 2 * num of platters * tracks * sectors * bytes per sector

$$\text{Size} = 2 * 4 * 8192 * 256 * 512 = 2^1 \cdot 2^2 \cdot 2^{13} \cdot 2^8 \cdot 2^9$$

$$\text{Size} = 2^{33} \text{ bytes} / (2^{10} \text{ bytes/KB}) / (2^{10} \text{ KB/MB}) / (2^{10} \text{ MB/GB})$$

$$\text{Size} = 2^{33} = 2^3 * 2^{30} = 8\text{GB}$$

Scale of Bytes

Multiple-byte units					V•T•E
Decimal			Binary		
Value	Metric		Value	IEC	Memory
1000	kB	kilobyte	1024	KiB kibibyte	KB kilobyte
1000 ²	MB	megabyte	1024 ²	MiB mebibyte	MB megabyte
1000 ³	GB	gigabyte	1024 ³	GiB gibibyte	GB gigabyte
1000 ⁴	TB	terabyte	1024 ⁴	TiB tebibyte	TB terabyte
1000 ⁵	PB	petabyte	1024 ⁵	PiB pebibyte	–
1000 ⁶	EB	exabyte	1024 ⁶	EiB exbibyte	–
1000 ⁷	ZB	zettabyte	1024 ⁷	ZiB zebibyte	–
1000 ⁸	YB	yottabyte	1024 ⁸	YiB yobibyte	–
1000 ⁹	RB	ronnabyte		–	
1000 ¹⁰	QB	quettabyte		–	
Orders of magnitude of data					

This is
what we
will use

More Disk Terminology

- Rotation Speed:
 - The speed at which the disk rotates (e.g, 5400RPM)

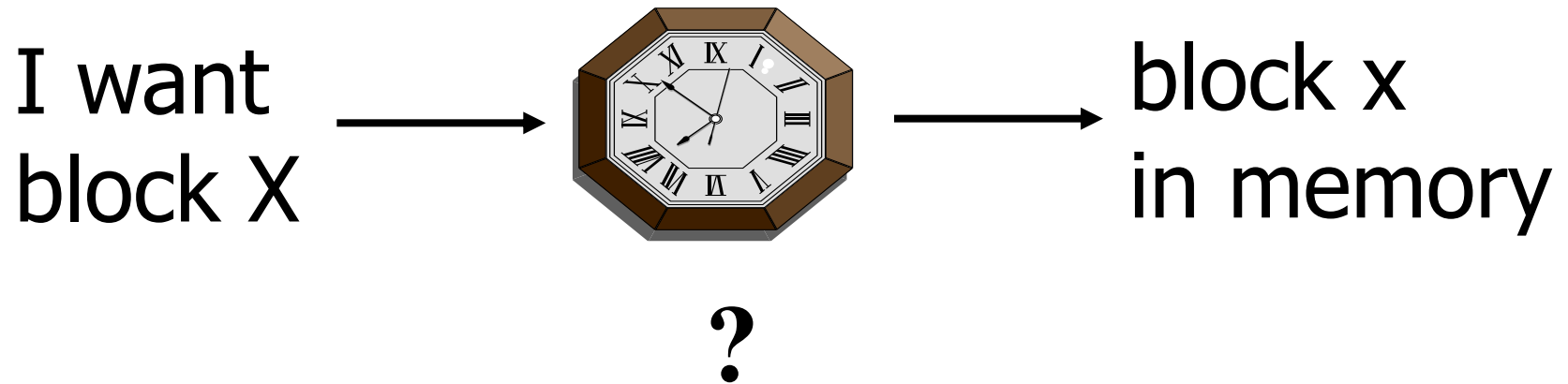
$$\text{maxlatency} = \frac{60}{\text{speed}}$$

$$\text{avglatency} = \frac{\text{maxlatency}}{2}$$

- Number of Tracks:
 - Typically, 10,000 to 15,000.
- Bytes per track:
 - $\sim 10^5$ bytes per track

5.56ms (11.12ms)

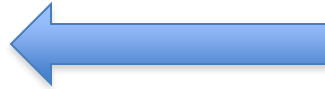
Big Question: What about access time?



$$\text{Latency} = \text{Disk Delay \{seek \& rotation\}} + \text{Transfer Time}$$

Transfer Time

Latency = Disk Delay
+ Transfer Time



- Typically, 100-200 MB/sec
- Reading 4K data block takes ~ 0.02-0.04 ms

Less than 1 millisecond

Disk Delay

Latency = Disk Delay ←
+ Transfer Time

More complicated

Disk Delay = Seek Time +
Rotational Latency

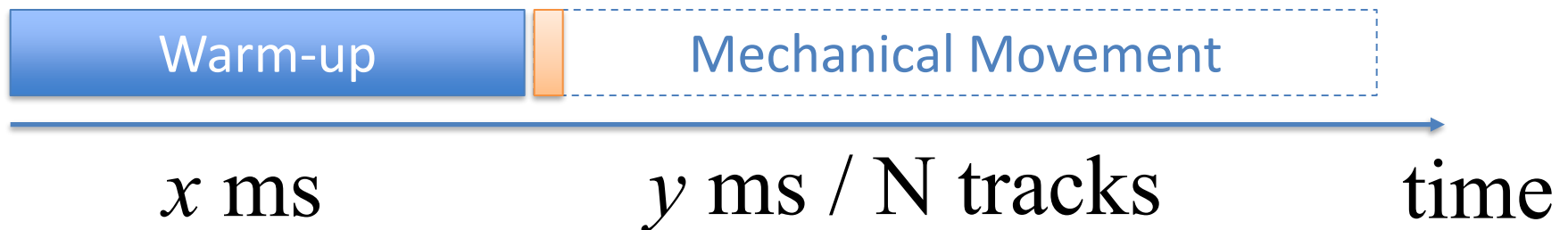
Average Seek Time

- Seek time is **most critical** time in Disk Delay.
- *Average* seek is about *travelling half the number of tracks on a surface*
- Average Seek Times:
 - Maxtor 40GB (IDE) ~10ms
 - Western Digital (IDE) 20GB ~9ms
 - Seagate (SCSI) 70 GB ~3.6ms
 - Maxtor 60GB (SATA) ~9ms
 - Seagate (SATA) 3 TB ~9ms

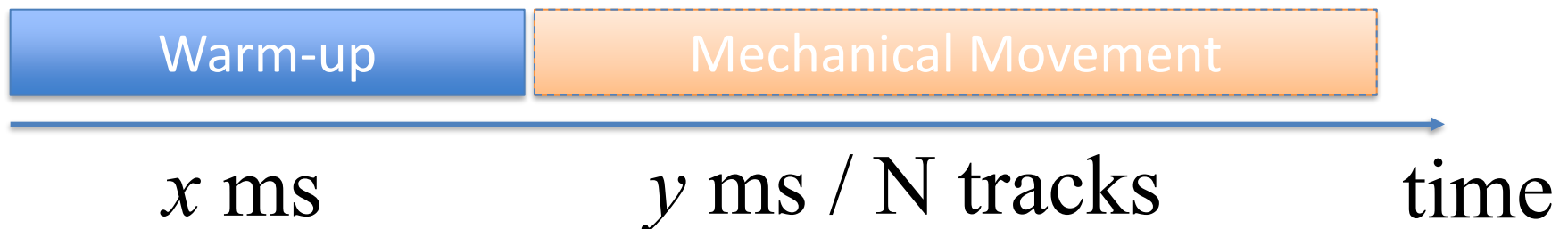
Order of 10 milliseconds

Breakdown of Seek Time

Move a few tracks



Move from outermost to innermost track



Breakdown of Seek Time

Move a few tracks



x ms

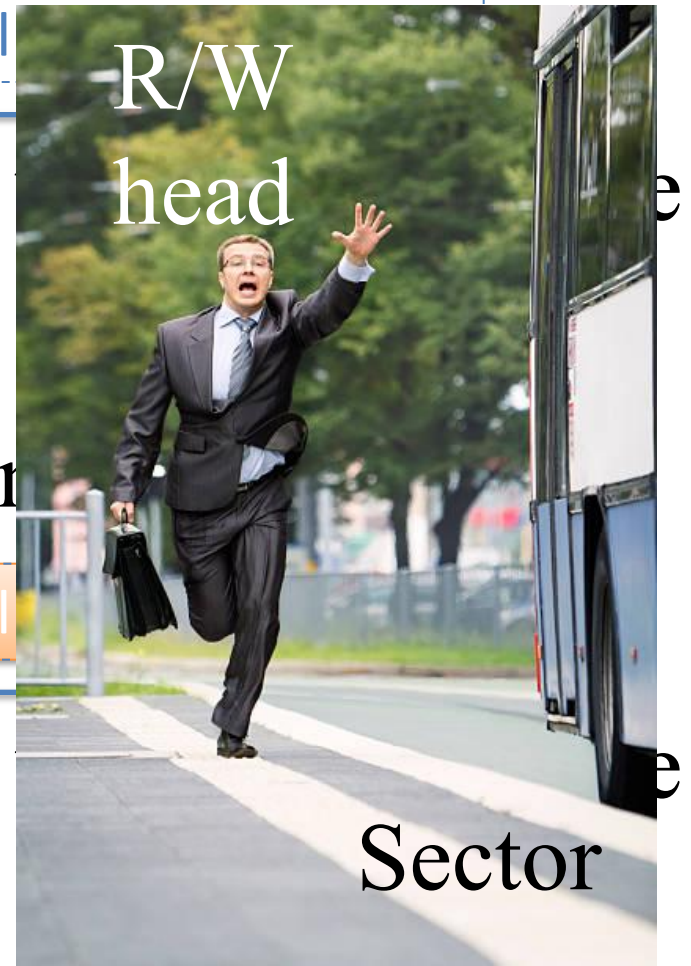
y ms / N

Move from outermost to inner

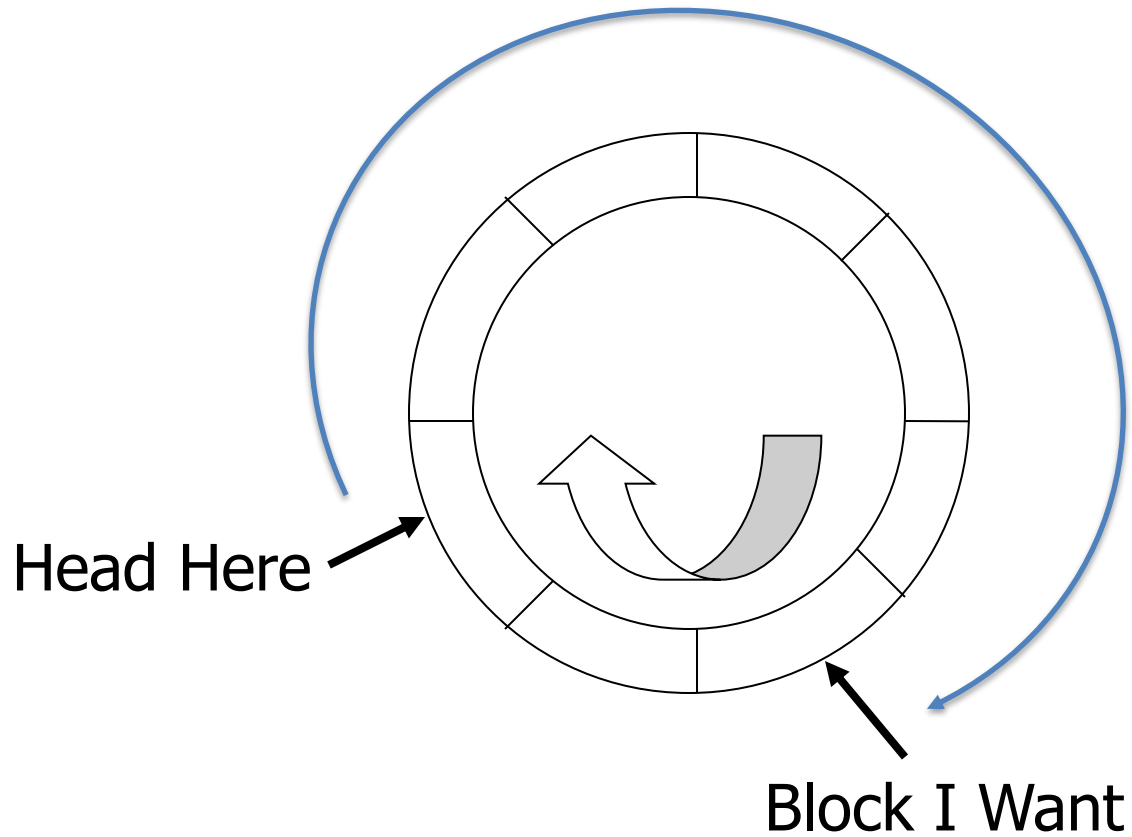


x ms

y ms / N



Rotational Latency



Average Rotational Latency

- Average latency is about half of the time it takes to make one full rotation.
- 3600 RPM = 8.33 ms
- 5400 RPM = 5.55 ms
- 7200 RPM = 4.16 ms
- 10,000 RPM = 3.0 ms (newer drives)

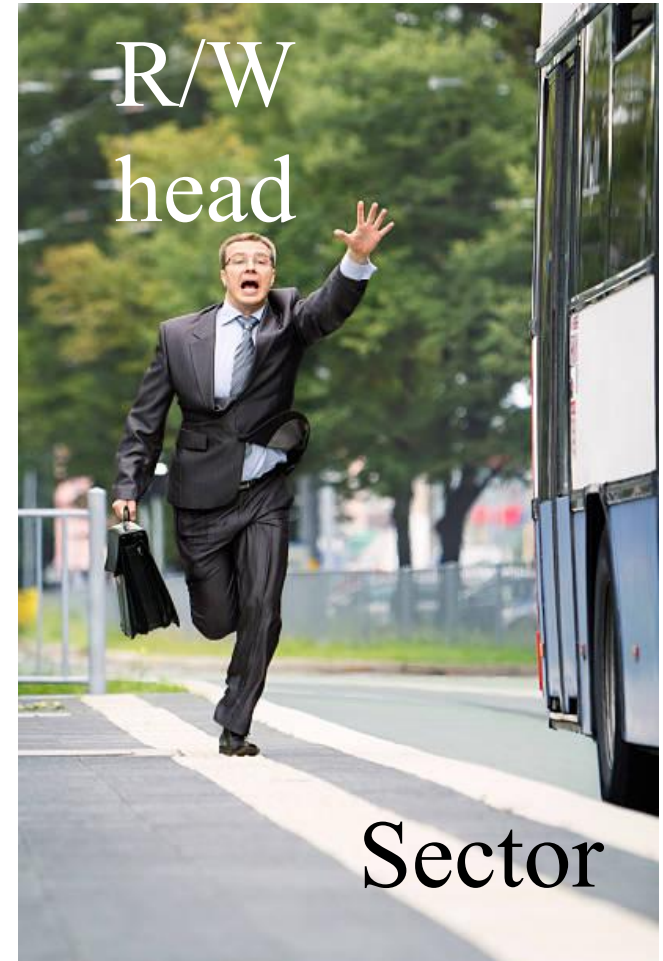
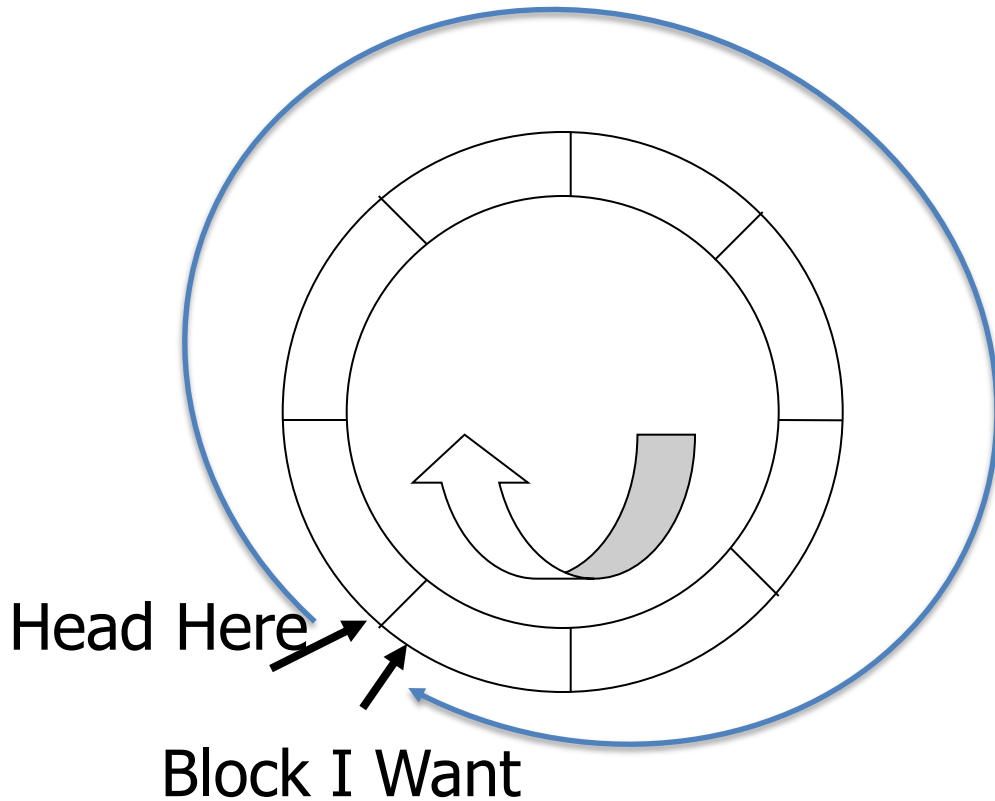
Order of few milliseconds

Example of HDD specs

$$\text{maxlatency} = \frac{60}{\text{speed}}$$

$$\text{avglatency} = \frac{\text{maxlatency}}{2}$$

Rotational Latency: worst case



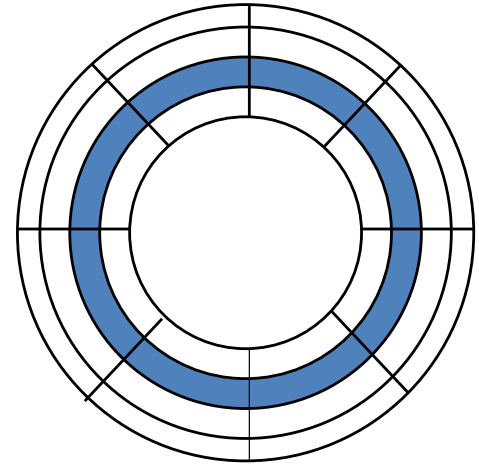
$$\text{maxlatency} = \frac{60}{\text{speed}}$$

Accessing a Disk Block: Summary

- Time to access (read/write) a disk block:
 - *seek time* (moving arms to position disk head on track)
 - *rotational latency* (waiting for block to rotate under head)
 - *transfer time* (actually moving data to/from disk surface)
- Key to lower I/O cost: **reduce seek/rotation latency! → sequential I/Os**

Example Disk Latency Problem

- Calculate the MIN, MAX & AVG disk latencies for reading a 4096-byte block on the same hard drive as before:



- 4 platters
- 8192 tracks
- 256 sectors/track
- 512 bytes/sector
- Disk rotates at 3840 RPM
- Seek time: 1 ms (warm-up), + 1ms for every 500 cylinders traveled.
- Reading one sector 0.06 ms

A 4096-byte block is 8 sectors

$$\frac{4096}{512}$$

The disk makes one rotation in (1/64 sec) = 15.6 ms

$$\frac{60}{3840}$$

Moving one track takes (1 + 0.002ms). Moving across all tracks takes (1 + 16.4ms)

$$\frac{1}{500} * 8192$$

22

Best Case: Minimum Latency

- Assume best case:
 - head is already on block we want!
- Here it's just read time of 8 sectors of 4096-byte block. We will pass over 8 sectors and 7 gaps.
- **That is only the “Transfer Time” $\approx 0.06 \text{ ms} \times 8 = 0.5 \text{ ms}$**

- 4 platters
- 8192 tracks
- 256 sectors/track
- 512 bytes/sector
- Disk rotates at 3840 RPM
- Seek time: 1 ms (warm-up), + 1ms for every 500 cylinders traveled.
- Reading one sector 0.06 ms

A 4096-byte block is 8 sectors

The disk makes one rotation in $(1/64 \text{ sec}) = 15.6 \text{ ms}$

Moving one track takes $(1 + 0.002\text{ms})$.
Moving across all tracks takes $(1 + 16.4\text{ms})$

Worst Case: Maximum Latency

- Now assume worst case:
 - The disk head is over innermost cylinder and the block we want is on outermost cylinder,
 - block we want has just passed under the head, so we have to wait a full rotation.

- 4 platters
- 8192 tracks
- 256 sectors/track
- 512 bytes/sector
- Disk rotates at 3840 RPM
- Seek time: 1 ms (warm-up), + 1ms for every 500 cylinders traveled.
- Reading one sector 0.06 ms

A 4096-byte block is 8 sectors

The disk makes one rotation in $(1/64 \text{ sec}) = 15.6 \text{ ms}$

Moving one track takes $(1 + 0.002\text{ms})$.
Moving across all tracks takes $(1 + 16.4\text{ms})$

14

= Warm-up + Time to move from innermost track to outermost track +
Time for one full rotation +
Time to read 8 sectors

= 1 (warm-up) + 16.4 ms (seek time) + 15.6 ms (one rotation) + 0.5ms (transfer time)

= **33.5 ms!!**

Average Case: Average Latency

- Now assume average case:
 - It will take an average amount of time to seek, and
 - block we want is $\frac{1}{2}$ of a revolution away from heads.

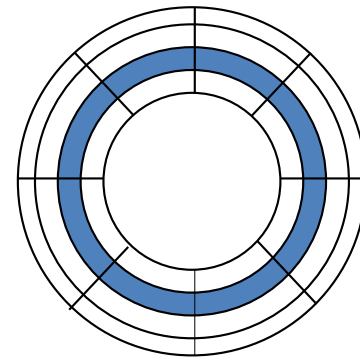
$$\begin{aligned}\text{Time} &= \text{Warm-up} + \text{Time to move over tracks} + \\ &\quad \text{Time for one-half of a rotation} + \\ &\quad \text{Time to read 8 sectors} \\ &= 1 + 8.2\text{ms (approximation)} + 7.8\text{ms (half rotation)} + \\ &\quad 0.5 \text{ ms (Transfer Time)} \\ &= \mathbf{17.5 \text{ ms}}\end{aligned}$$

Writing Blocks

- Same as reading blocks ...

After seeing all of this ...

- Which will be faster ***Sequential I/O*** or ***Random I/O***?



- **Sequential I/O**
 - Reading/writing blocks next to each other on the same track (or cylinder)

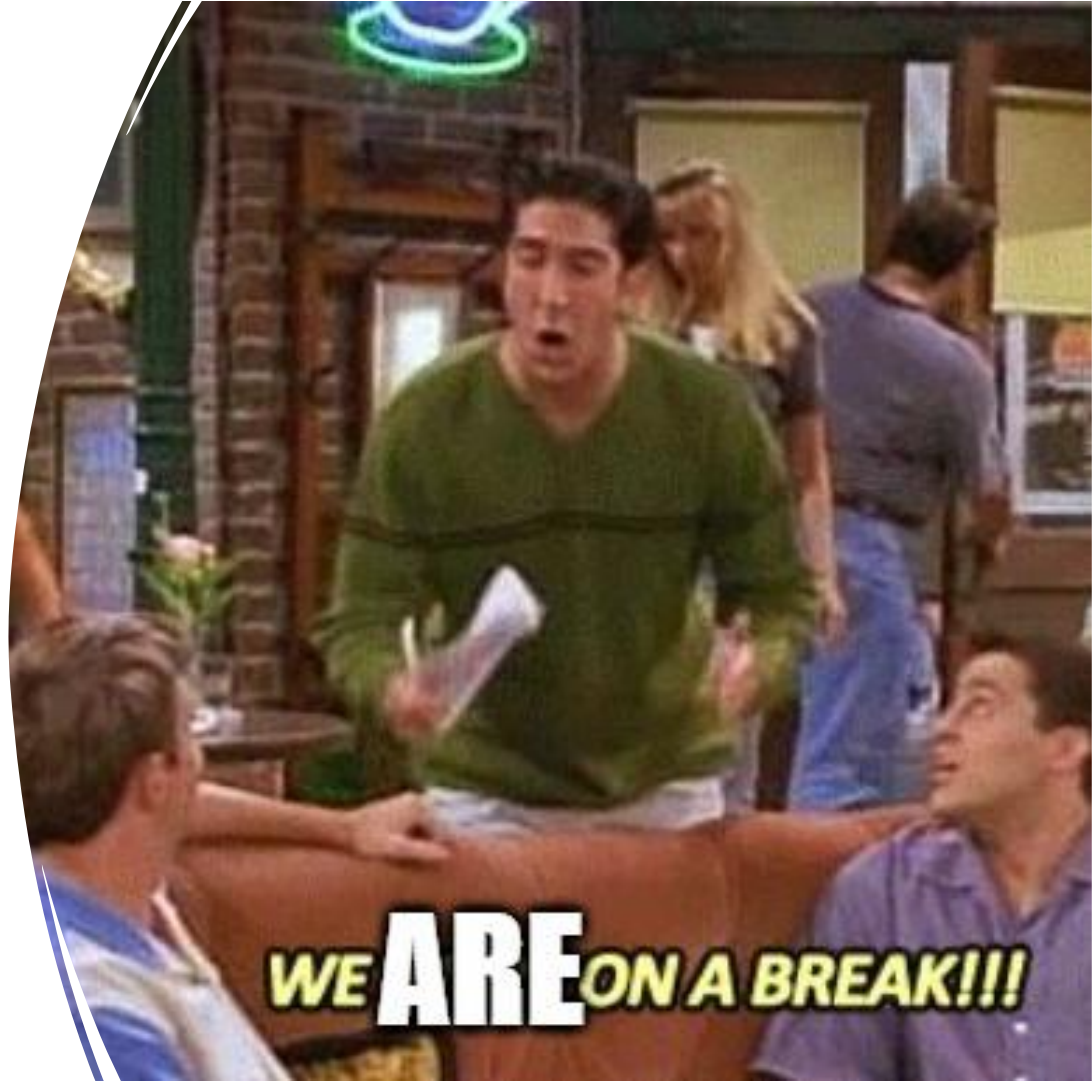
Sequential I/O saves seek & rotation times

Summary

- Practice how to compute:
 - Disk size
 - Transfer time
 - Seek time
 - Latency (rotation) time
- **Practice best/worst/average cases**

Break

- Now we will take a few minutes break



CS4432: Database Systems II

Data Storage

Accelerating Access to Blocks



DreamStudio Lite prompt:
velocimeter insider car
dashboard showing high speed

Learning goals

- Explain main techniques for accelerating access to blocks, & their pros/cons
- Execute SCAN algorithm for disk scheduling
- Contrast sequential read times for single and double buffering

Accelerating Access to Blocks

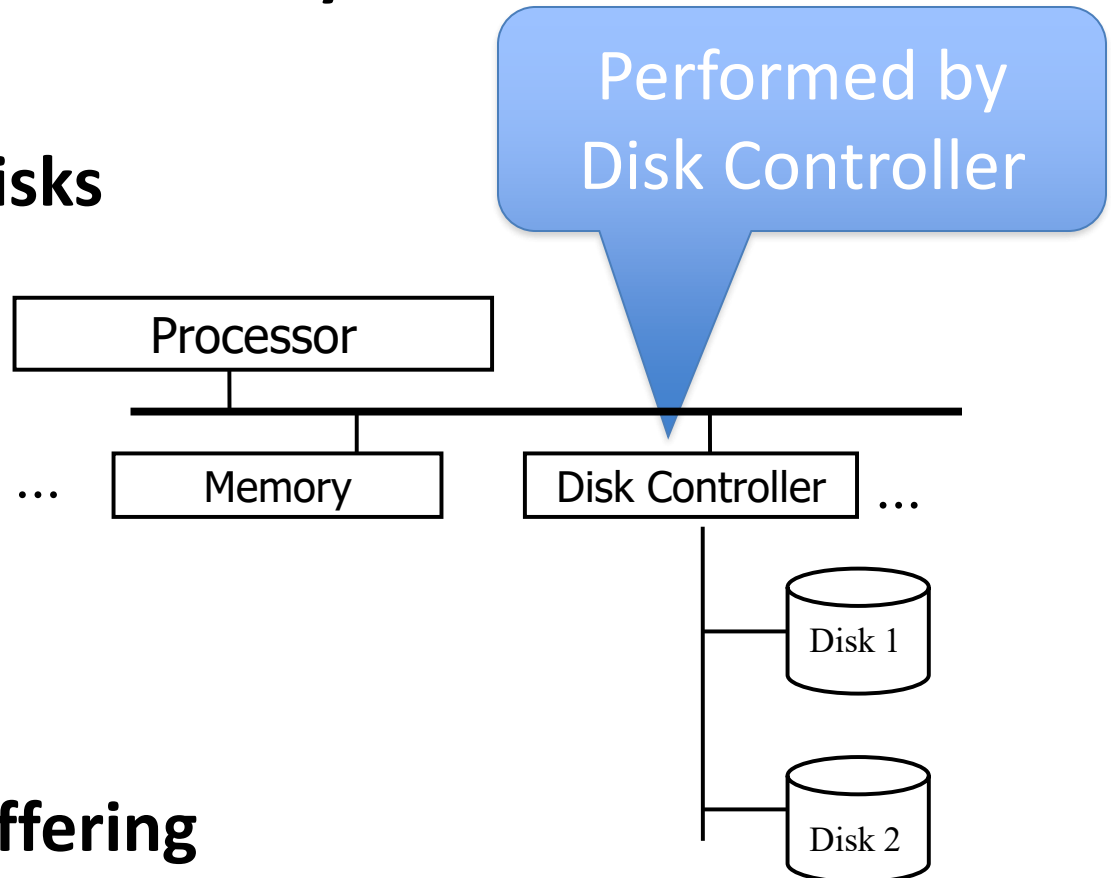
1. Placing Related Blocks on Cylinders

2. Using Multiple Disks

3. Mirroring

4. Disk Scheduling

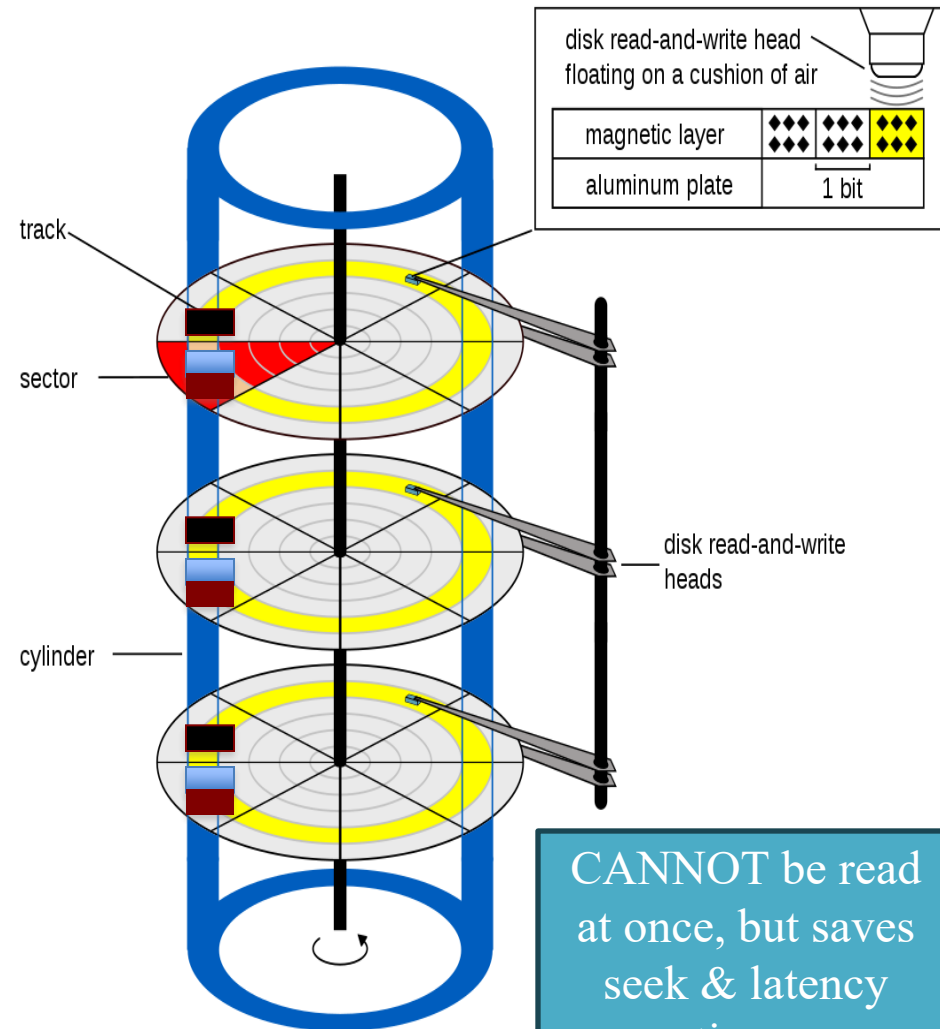
5. Prefetching & Buffering



#1: Placing Related Blocks on Cylinders

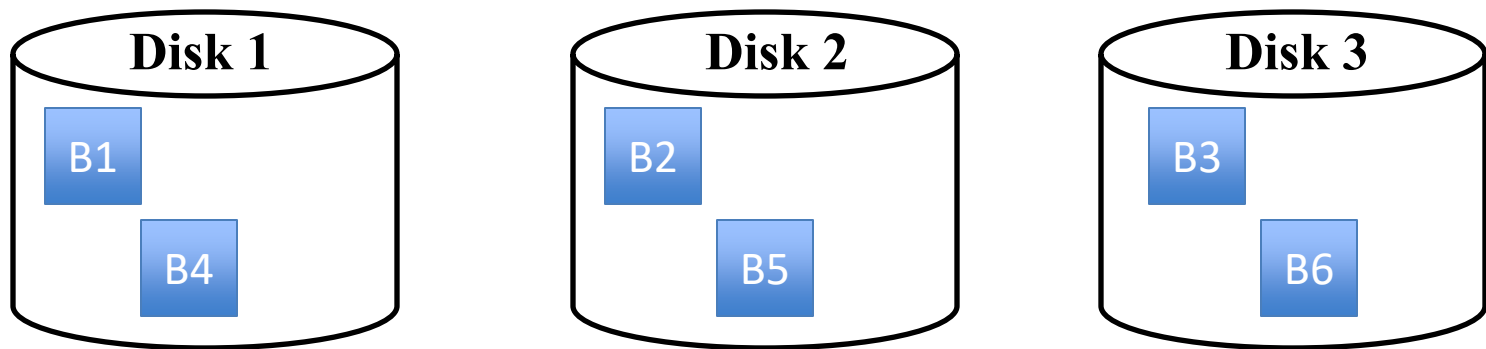
- Start by adding blocks on the next sectors on the same track
- Put on the same cylinder to save seek & rotation times
- Make use of the 2 surfaces in the same cylinder

- Put on the next sector on same track
- Put on the top surface in same cylinder
- Put on the bottom surface in same Cylinder as the blue ones



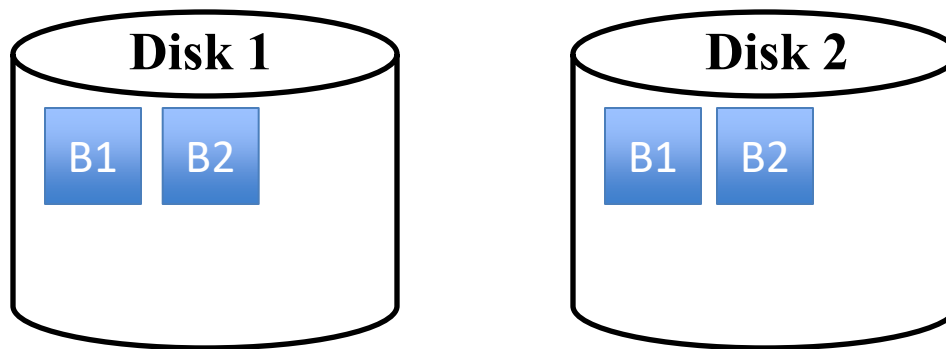
2- Using Multiple Disks: Striping

- Use multiple smaller disks instead of one large disk
- Each disk can access its data independently
 - N disks → N times faster access



3- Mirroring

- Use pairs of disks that are mirrors to each other
- Good for failure & Good for faster access
- Higher overhead under writing operations



4- Disk Scheduling

- Disk Controller may have a sequence of ***block requests*** (***queue of requests***)
- Not necessary serve requests in their arrival order (FIFO) → Use better scheduling policy
- ***Elevator (SCAN)*** policies

Example

Order of
receiving
requests



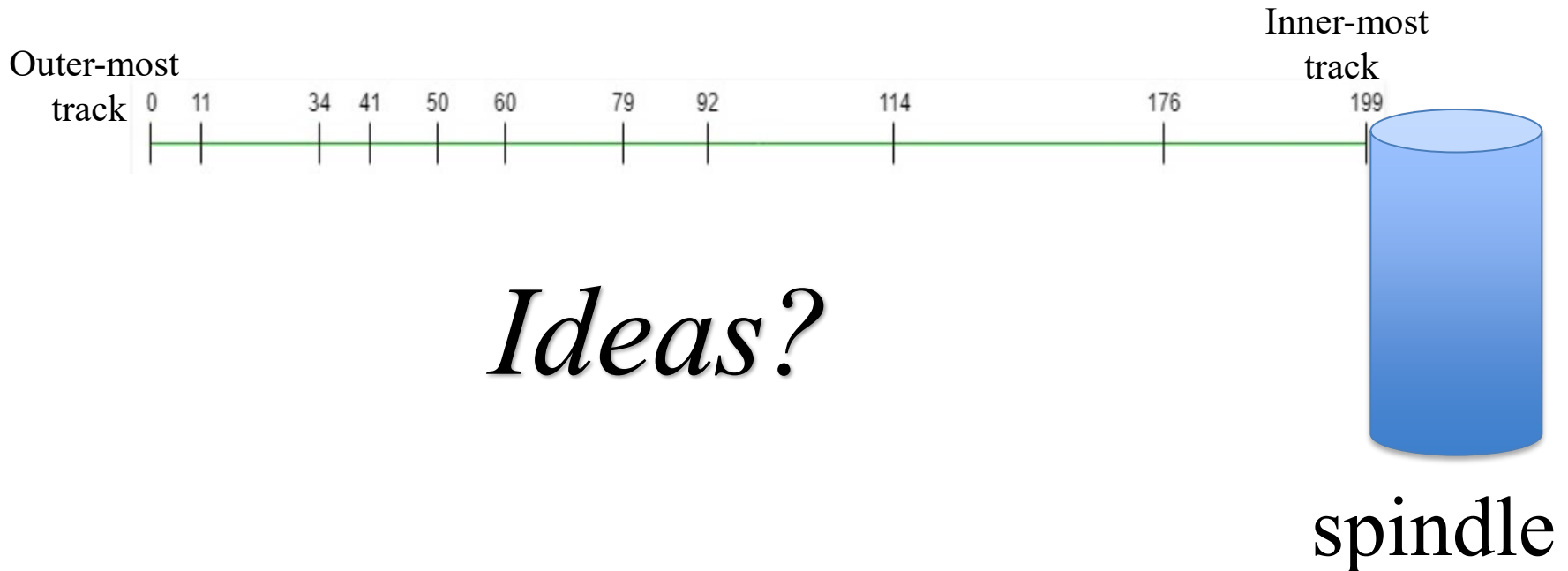
23, 89, 132, 42, 187

Cylinder #



Initial position (time = 0): #100

Direction (time = 0): smaller numbers (towards outer-most track)



4- Disk Scheduling: SCAN

- **Input**

- Current position of the head (track number)
- Direction: left (outer), or right (inner)
- queue of requests

aka “Elevator”
algorithm

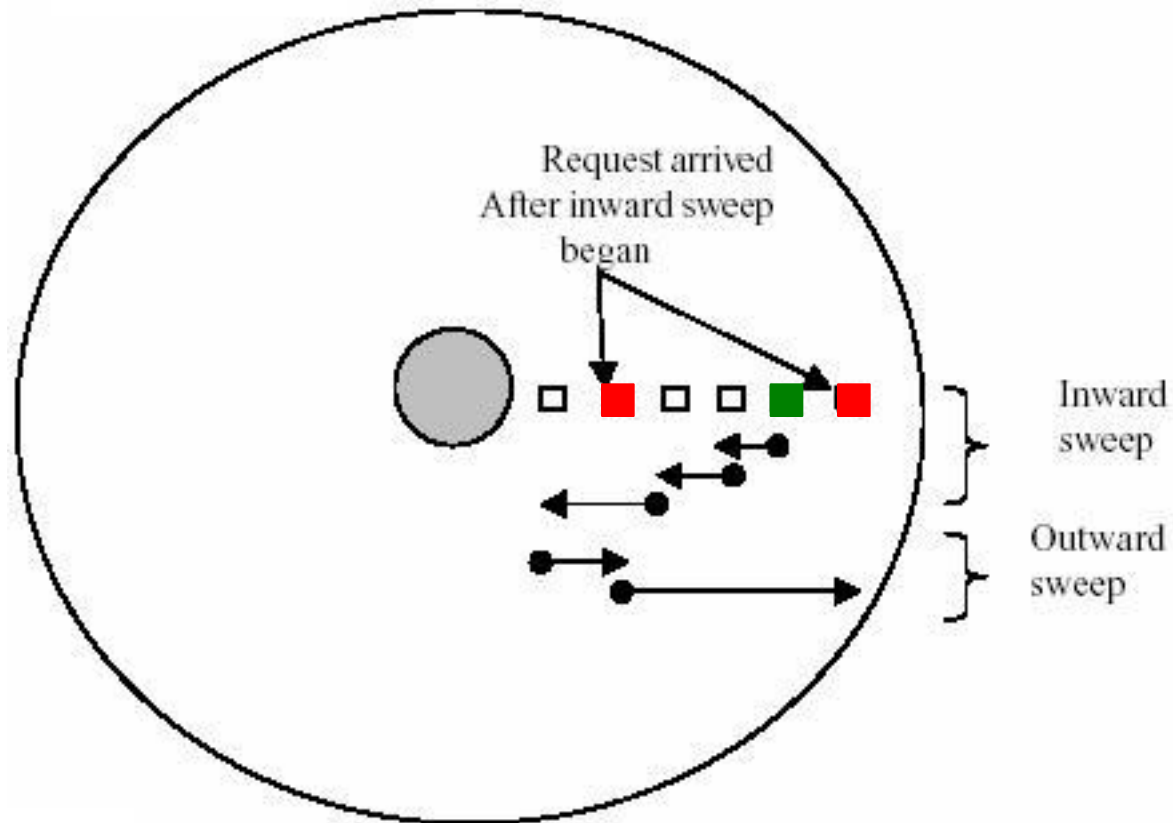
- **Algorithm**

- Move in the decided direction
- Serve requests in the order of seeing them
- Reach the end of the disk in this direction
- Reverse direction
- Serve remaining requests in the order seeing them

- **What ‘seeing’ means?**

- If request for track X arrived by the time the head reaches X, **it is served**
It is not directly related to the arrival order.

4- Disk Scheduling: SCAN



- **When starting a sweep (inward or outward)**
 - Complete the sweep until the end, processing requests “seen” along the way

Example

Order of
receiving
requests

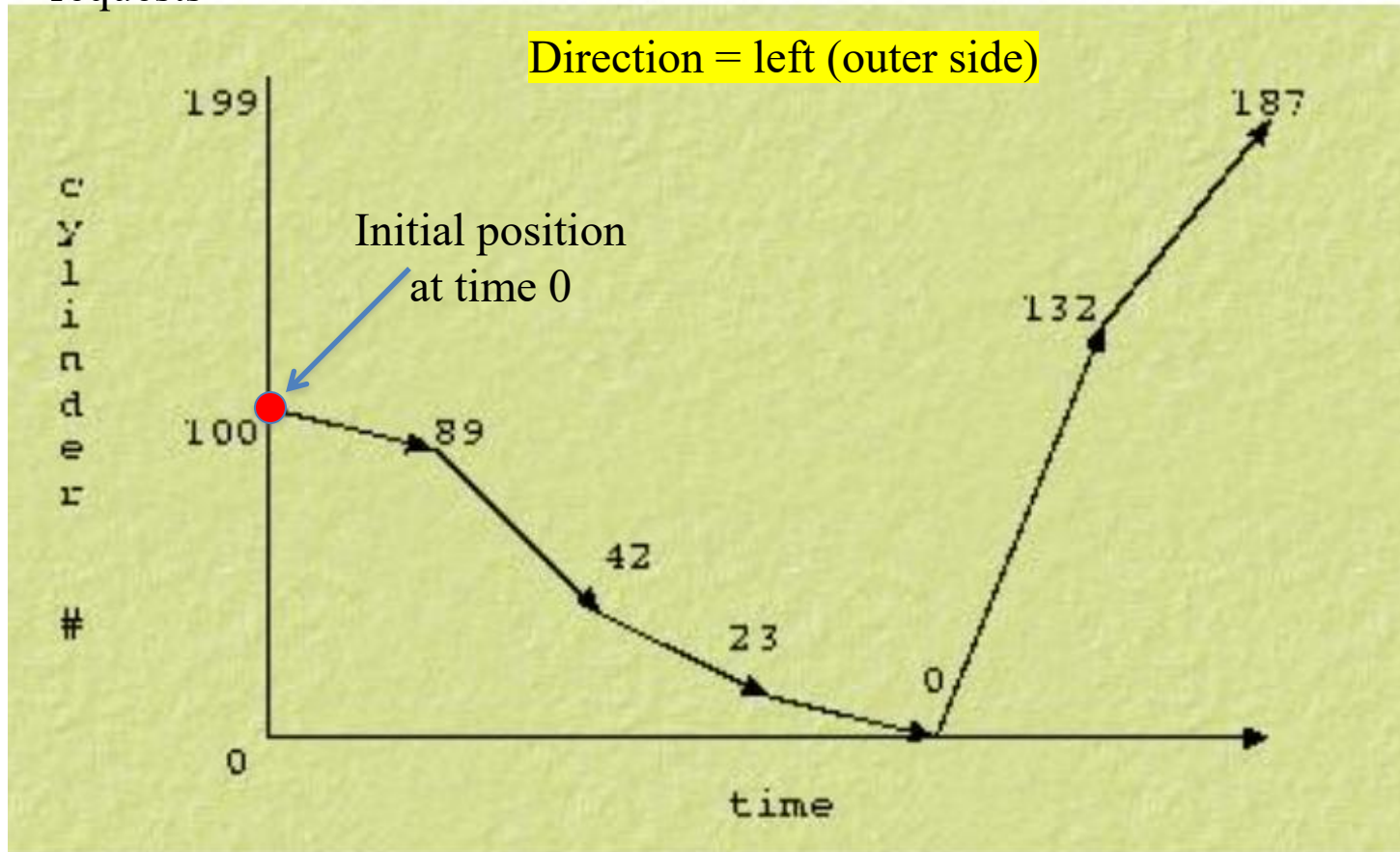


23, 89, 132, 42, 187

Cylinder #



Direction = left (outer side)



$$11+47+19+23+132+55=287$$

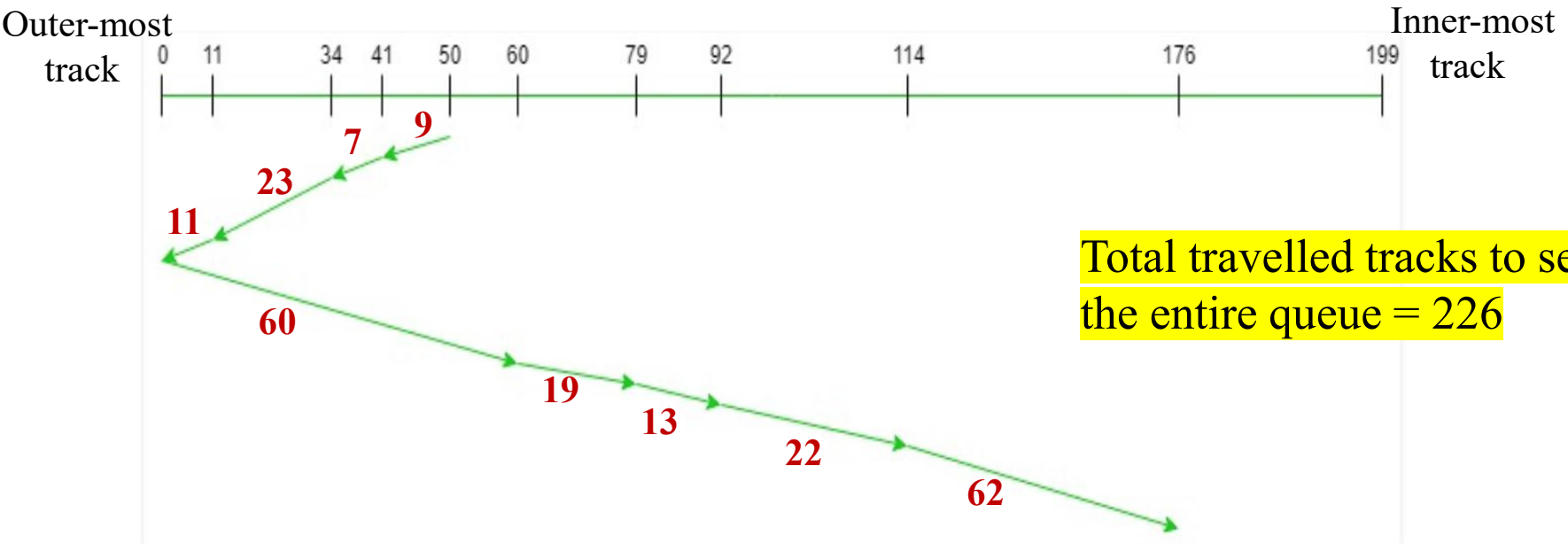
(Another) Example

Input:

Request sequence = {176, 79, 34, 60, 92, 11, 41, 114}

Initial head position = 50

Direction = left (We are moving from right to left)



5- Prefetching & Buffering

- If DBMS can predict the sequence of access
 - It can pre-fetch and buffer more blocks even before requesting them.

Example: **Have a File**

» Sequence of Blocks B1, B2, ...

Have a Program

» Process B1

» Process B2

» Process B3

...

Naïve Single Buffer Solution

- (1) Read B1 → Buffer
- (2) Process Data in Buffer
- (3) Read B2 → Buffer
- (4) Process Data in Buffer

...

Cost of Naïve Solution

Say P = time to process a block (after being in memory), e.g., updating its data

R = time to read in a block disk (sum of seek + latency + transfer)

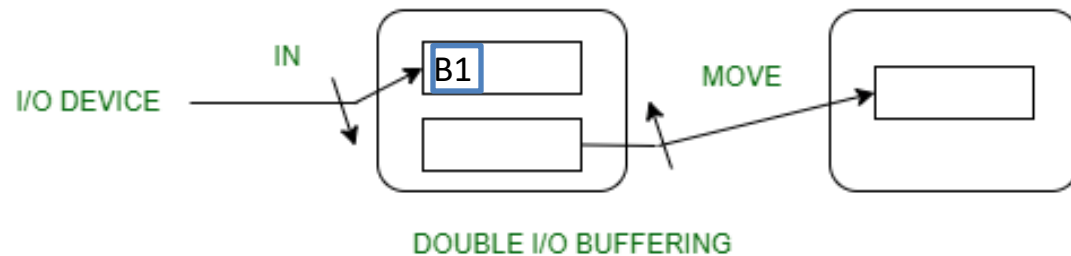
n = # blocks

Using Single buffer

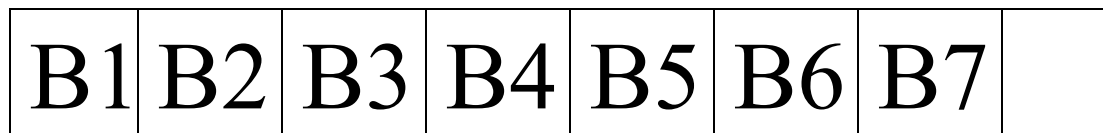
Total time needed = $n(P+R)$

Prefetching & Buffering

Memory:

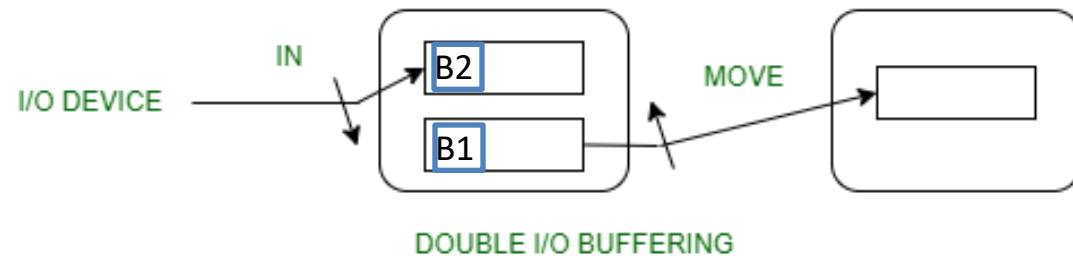


Disk:

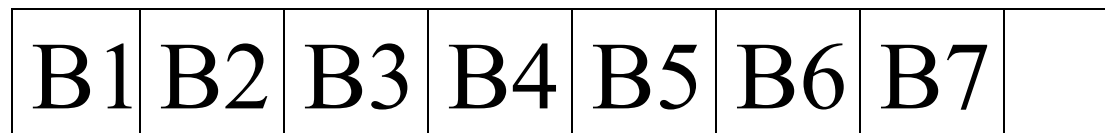


Prefetching & Buffering

Memory:

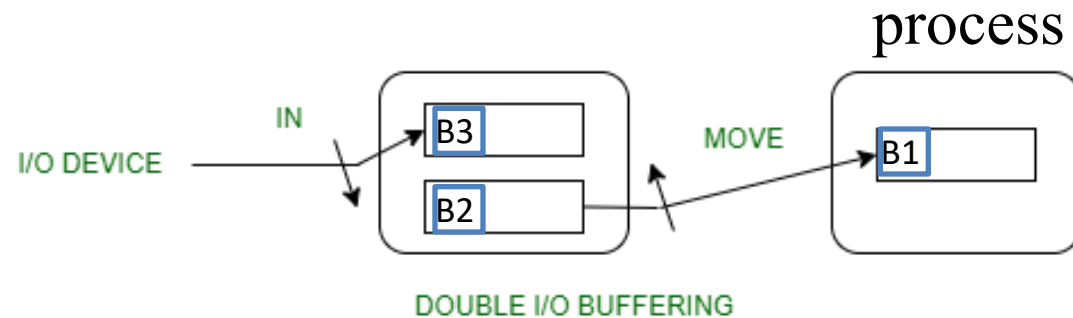


Disk:

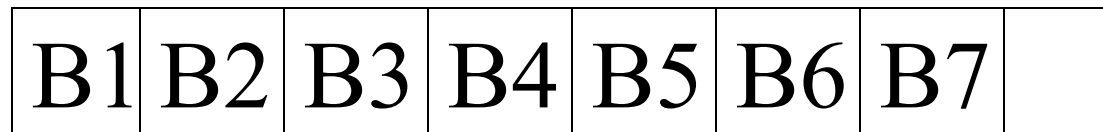


Prefetching & Buffering

Memory:

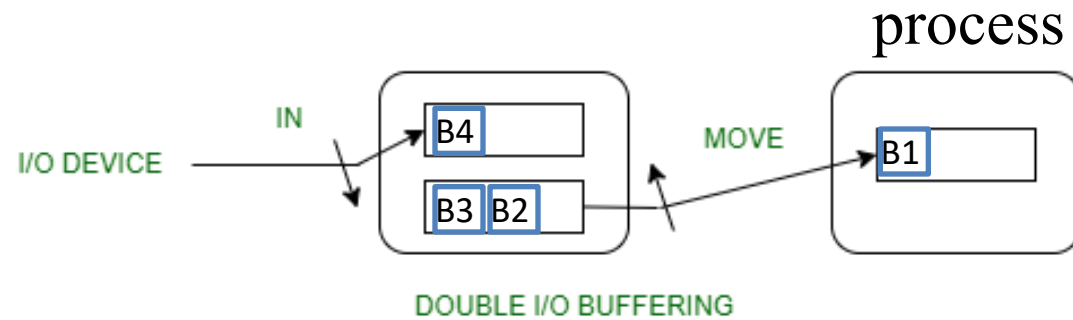


Disk:

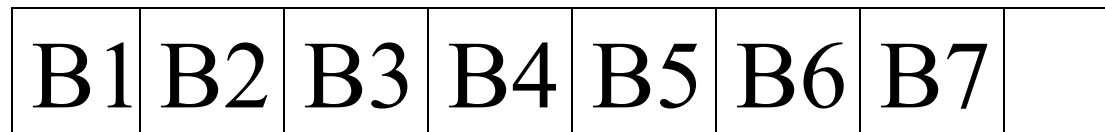


Prefetching & Buffering

Memory:

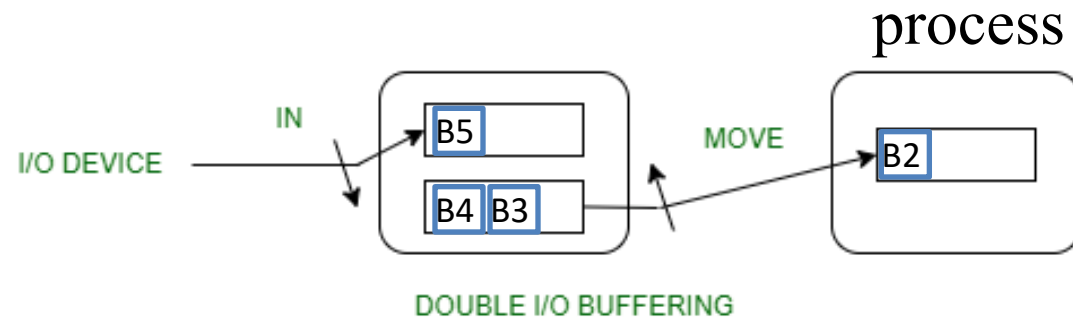


Disk:

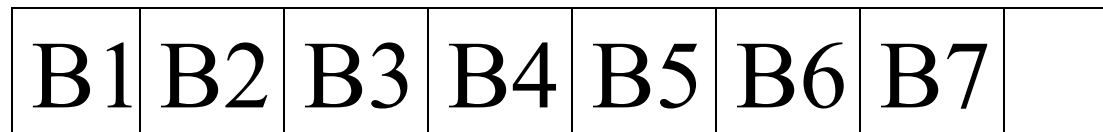


Prefetching & Buffering

Memory:

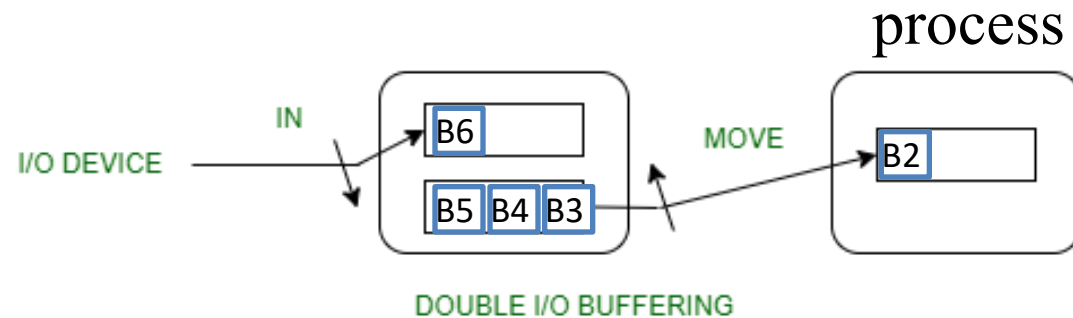


Disk:

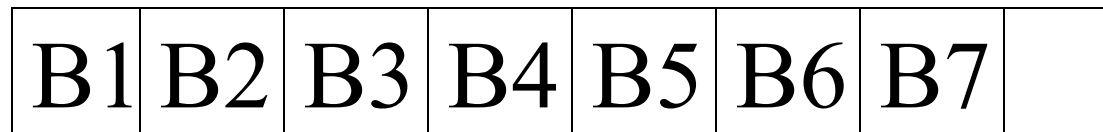


Prefetching & Buffering

Memory:



Disk:



Cost of Double Buffering

P = Processing time/block

R = IO time/block

n = # blocks

- In Double Buffering
 - Have to pay R for the first block
 - Assuming sequential I/O, time to read other blocks is very small compared to P
 - Time to retrieve them overlaps with nP

Cost of Double Buffering

P = Processing time/block

R = IO time/block

n = # blocks

- Double Buffering time = $R + nP$
- Single Buffering time = $n(R+P)$

Accelerating Access to Blocks: Summary

1. **Placing Related Blocks on Cylinders**
2. **Using Multiple Disks**
3. **Mirroring**
4. **Disk Scheduling**
5. **Prefetching & Buffering**

